1. Purpose and Scope

This document explains the main technical decisions for AITU Booking. It covers four groups of decisions:

1. Mobile clients (iOS and Android)
2. Backend framework
3. Database
4. Availability updates (real-time status delivery)

The technology for the administrator web panel is outside the scope of this document and will be selected separately.

2. Project Requirements

The choices are evaluated against the following requirements. Each one has an identifier (R1–R7), which is used in the comparison tables.

ID	Requirement
R1	Provide an iOS application, an Android application, and an administrator web panel.
R2	Prevent overlapping room bookings, including bookings submitted at the same moment.
R3	Check room availability against the university timetable.
R4	Update availability on connected devices without manual refresh.
R5	Import the timetable from PDF and allow manual corrections.
R6	Store users, rooms, consultation slots, bookings, and key records consistently.
R7	Allow bookings only within a window of 1 to 72 hours ahead.

3. Comparison Criteria

The same four criteria are used in every group:

Criterion	What is assessed
Complexity	The amount of work and the number of difficult design decisions needed to implement the project's features.
Requirements support	How directly the technology supports R1–R7, without workarounds.

Team skills	Whether the team can work with the technology now or must learn a new language or model.
Deployment	What is needed to build, release, and run the component.

Team context (to be confirmed by the team): the backend developers work in Python. The iOS developer works in Swift, and the Android developer works in Kotlin. The "Team skills" rows below are based on these assumptions.

Performance has not been measured yet. The comparison therefore relies on documented capabilities and does not include benchmark results.

4. Mobile Clients

Criterion	Native (Swift + SwiftUI, Kotlin + Compose)	Flutter	React Native
Complexity	Two codebases; each screen is built twice	One codebase; platform-specific features need plugins	One codebase; missing features need native modules
Requirements support	Built-in WebSocket clients and app lifecycle events for reconnection (R4)	Supported through Dart packages	Supported through JavaScript APIs
Team skills	Matches current Swift and Kotlin skills	Requires Dart, not used elsewhere	Requires TypeScript and React, not used elsewhere
Deployment	Standard App Store and Google Play process	Same stores plus the Flutter toolchain	Same stores plus the Node.js toolchain

Selected: Swift + SwiftUI for iOS and Kotlin + Jetpack Compose for Android.

The project needs separate iOS and Android applications (R1). A cross-platform framework would save effort on only two of the three clients, because the administrator panel is a separate web codebase anyway. Real-time availability (R4) depends on reliable reconnection and on reloading data when the app returns to the foreground, and native APIs provide this directly, without a third-party bridge. The team already works in Swift and Kotlin, while a cross-platform framework would add a language that no other part of the project uses.

The trade-off is that the user interface is written twice. To keep the two apps consistent, the booking rules (R2, R3, R7) are enforced only on the server, and the clients only display the results.

5. Backend Framework

Criterion	FastAPI	Django REST Framework	NestJS
Complexity	Lightweight; the team organises the module structure	Ready-made auth and ORM; workflows must fit Django's structure	Prescribed modular structure with dependency injection
Requirements support	Built-in WebSockets (R4); PDF parsing in the same codebase (R5)	WebSockets need Django Channels (R4); Python PDF parsing (R5)	WebSockets available (R4); PDF parsing needs JS libraries or a separate service (R5)
Team skills	Python, already used	Python, plus Django conventions	Requires TypeScript
Deployment	One ASGI process (Uvicorn)	WSGI, or ASGI once Channels is added	One Node.js process

Selected: FastAPI.

PDF timetable import (R5) is planned in Python, so with FastAPI it runs inside the same backend rather than in a separate service. WebSocket support (R4) is part of the framework, while Django would need the additional Channels package. Request validation and generated OpenAPI documentation give the iOS, Android, and web clients one shared API contract (R1). The team already works in Python.

FastAPI does not prescribe a project structure, so the backend will start as one application with modules for authentication, rooms, bookings, consultations, timetable import, and key tracking.

6. Database

Criterion	PostgreSQL	MySQL (InnoDB)	MongoDB
Complexity	Overlap prevention is declared in the schema	Overlap prevention is written in code with locking reads	Document modelling decisions; overlap checks written by hand

Requirements support	Exclusion constraint rejects overlapping bookings (R2); foreign keys (R6)	Correct only if every code path locks the same way (R2); foreign keys (R6)	Explicit conflict checks (R2); relations not enforced (R6)
Team skills	SQL, plus range types and exclusion constraints	SQL	MongoDB query language
Deployment	Single server or managed instance	Single server or managed instance	Transactions require a replica set

Selected: PostgreSQL.

The main reason is R2. An exclusion constraint (`EXCLUDE USING gist (room_id WITH =, period WITH &&)` with the `btree_gist` extension) makes the database itself reject overlapping bookings for the same room, even if two requests arrive at the same moment or a code path omits a check. The project data is relational (R6), and the 1–72-hour window (R7) is checked in the booking transaction.

An exclusion constraint on the bookings table does not see classes stored in a separate timetable table (R3). This will be solved either by storing bookings and classes in one "room occupancy" table covered by a single constraint, or by locking the affected room in one transaction during booking creation and timetable publication. In both cases, a newly imported timetable that conflicts with existing bookings is reported to the administrator rather than overwriting them.

7. Availability Updates

Booking creation, cancellation, and timetable upload use REST in every option. The options differ only in how clients learn that availability has changed (R4).

Criterion	WebSocket + REST	Server-Sent Events + REST	Periodic polling
Complexity	Connection auth, reconnection, event handling	Event stream with reconnection	Timer and repeated requests
Requirements support	Instant updates (R4)	Instant updates (R4); one-way is sufficient	Delay up to one polling interval (R4 partly met)

Team skills	Built-in clients on iOS, Android, and FastAPI	No built-in client on iOS or Android	Uses the existing REST client
Deployment	Proxy must support WebSocket upgrade	Proxy must disable response buffering	No special infrastructure

8. Summary

- **iOS client:** Swift + SwiftUI (R1, R4, team skills)
- **Android client:** Kotlin + Jetpack Compose (R1, R4, team skills)
- **Backend:** FastAPI (R4, R5, Python)
- **Database:** PostgreSQL (R2, R6)
- **Availability updates:** WebSocket + REST (R4)

9. Validation Plan

1. Send simultaneous booking requests for the same room and interval, and verify that only one booking is saved (R2).
2. Test bookings at the boundaries of the 1–72-hour window (R7).
3. Import a timetable that conflicts with an existing booking, and verify that the conflict is reported (R3, R5).
4. Measure the time between a database change and its appearance on another device (R4).
5. Compare WebSocket updates with a polling baseline under the same workload (R4).
6. Disconnect and reconnect a client, and verify that it receives the current availability (R4).

These checks are planned. No performance results or completed test outcomes are claimed at this stage.

References

1. Apple — SwiftUI
2. Android — Jetpack Compose
3. Flutter — Documentation
4. React Native — Documentation
5. FastAPI — Features
6. FastAPI — WebSockets
7. Django REST Framework — Documentation
8. Django Channels — Documentation
9. NestJS — Documentation
10. PostgreSQL — Range Types
11. PostgreSQL — btree_gist
12. MySQL — InnoDB Locking Reads
13. MongoDB — Transactions

14. MDN — WebSocket API
15. MDN — Server-Sent Events